



Classifier Ensemble

Outline

- Why Ensemble?
- Components in an ensemble
- Ensemble Based Systems
 - Bagging
 - Boosting
 - AdaBoost
- How much classification improvements with an ensemble?

Why Ensembles Work (Key Theory)

- **Goal:** reduce **generalization error**
- **Error=Bias²+Variance+Noise**

For a model $\hat{f}(x)$ trained on dataset D :

$$\mathbb{E}_D[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}_D[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_D[(\hat{f}(x) - \mathbb{E}_D[\hat{f}(x)])^2]}_{\text{Variance}} + \text{Noise}$$

*how far the average prediction
is from the true function*

*how much predictions
fluctuate across datasets*

Bias $\rightarrow$ *systematic error*

Model too simple, e.g., linear model for nonlinear data

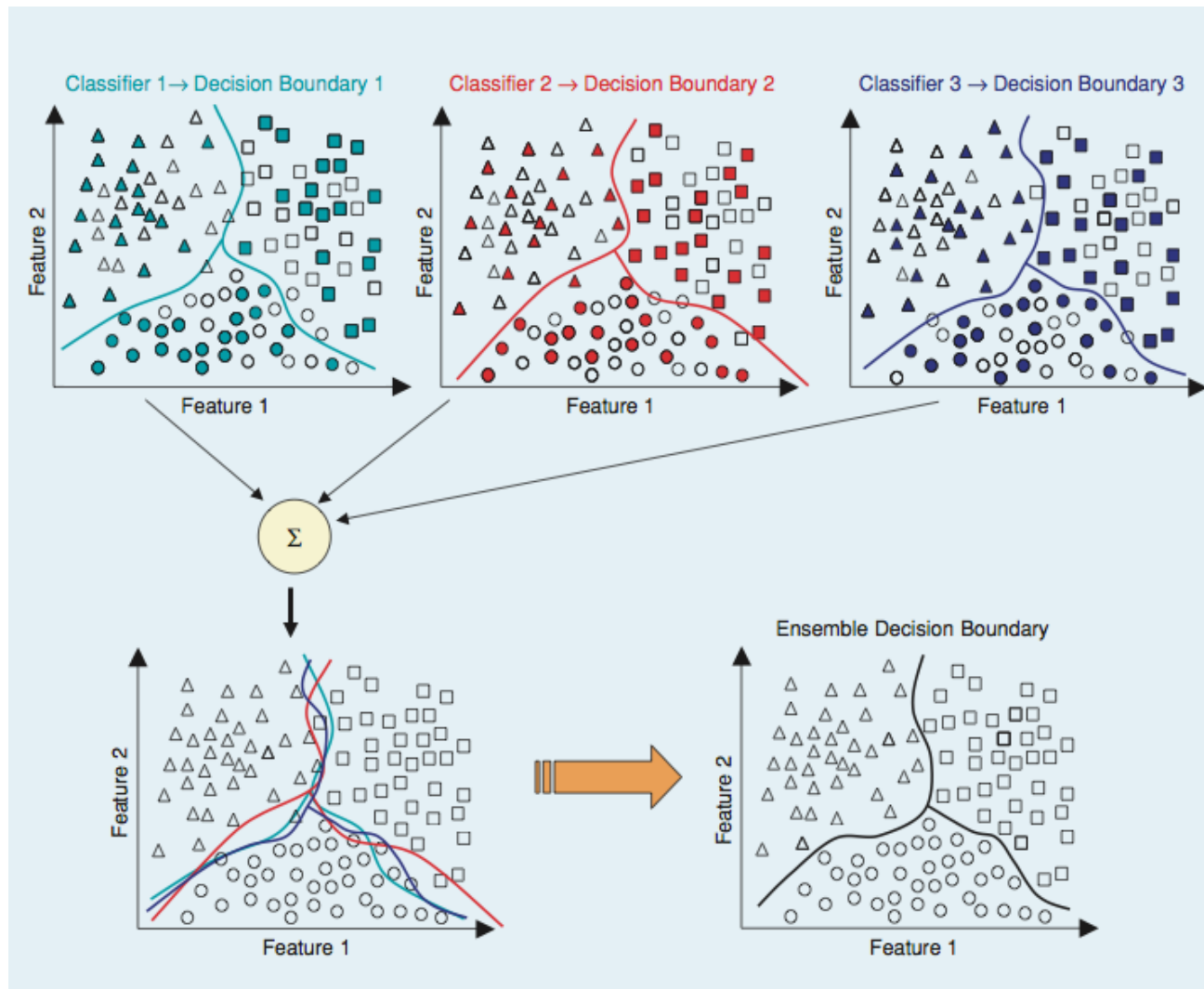
Variance $\rightarrow$ *instability*

Model changes a lot with small data changes, decision tree splits differently

Why Ensembles Work (Key Theory)

- Two mechanisms:
 - **Variance reduction** → Bagging / Random Forest
 - **Bias reduction** → Boosting
- Making important decisions
 - Expert panel, Lifeline (opinion of expert “friends”)

Components in an Ensemble



Train many models:

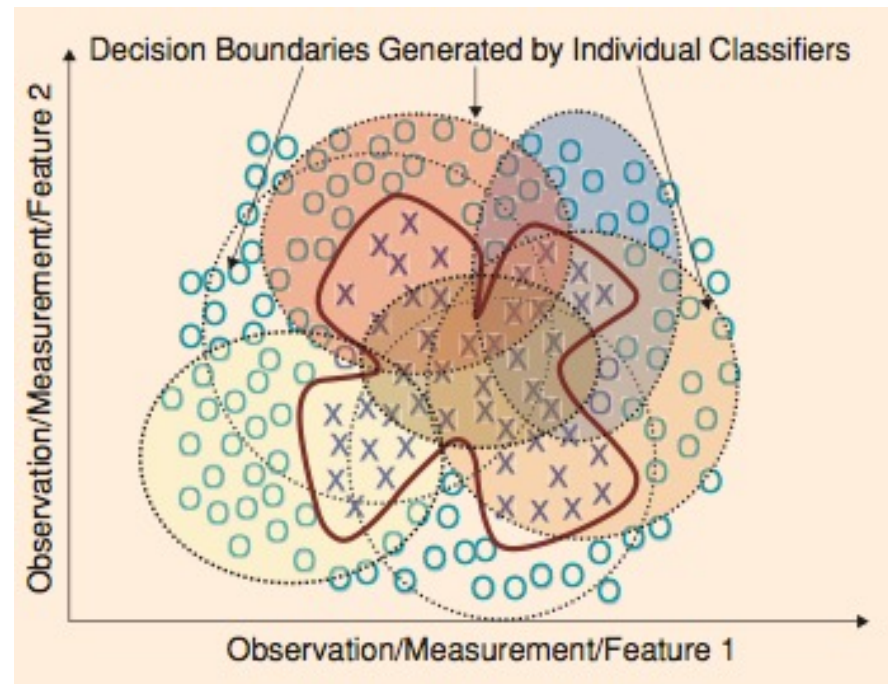
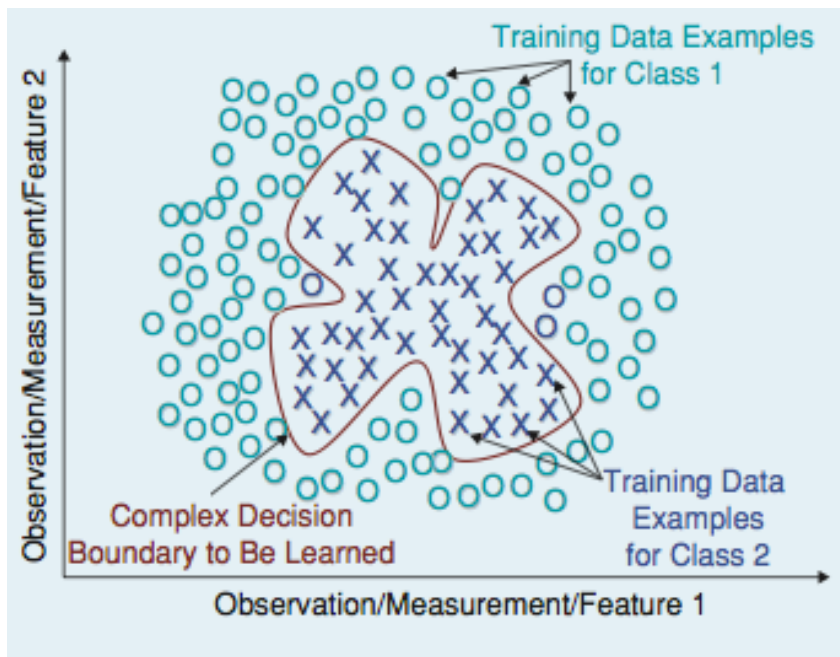
$$\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_T(x)$$

Final prediction:

$$\hat{f}_{ensemble}(x) = \frac{1}{T} \sum_{t=1}^T \hat{f}_t(x)$$

Why Ensemble?

- Divide and Conquer
- Each model learns **different decision boundary**
- Aggregation smooths irregular boundaries



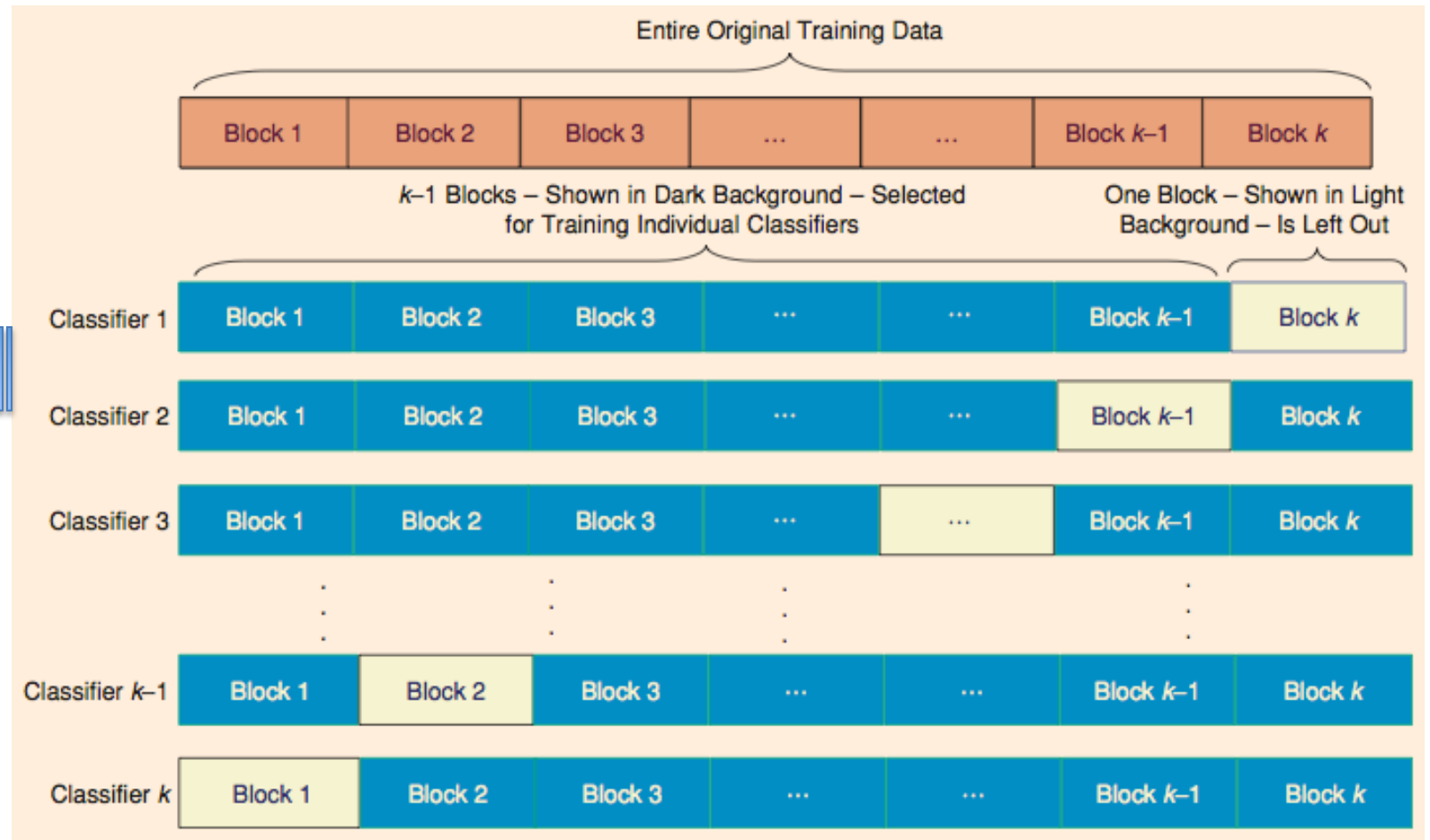
What Makes a Good Ensemble?

- Two requirements:
 - **Accuracy** (better than random)
 - **Diversity** (make different errors)
- If all models make the same mistakes → ensemble fails
 - Build an ensemble of classifiers **as diverse as** possible

Base Classifiers

- Good candidate base classifiers:
 - Decision Tree (most widely used)
 - Logistic Regression, (Naïve) Bayes, KNN
 - SVM, Neural Networks
 - SVM and Neural Networks are **less commonly used**
 - already strong learners, higher training cost, less benefit from boosting
- In practice:
 - Bagging / Random Forest → **Decision Trees dominate**
 - Boosting → **Stumps / shallow trees dominate**

K-fold data splitting



**Majority
Vote**



Not true ensemble method

Bagging

- Main idea:
 - Form different training data to train each classifier
 - How?
 - Create bootstrapped training data-- randomly pick a certain percent of data from the original data **with replacement**
 - The classifiers are combined with majority vote

Bagging

Algorithm: Bagging

Input:

- Training data S with correct labels $\omega_i \in \Omega = \{\omega_1, \dots, \omega_C\}$ representing C classes
- Weak learning algorithm **WeakLearn**,
- Integer T specifying number of iterations.
- Percent (or fraction) F to create bootstrapped training data

Do $t = 1, \dots, T$

1. Take a bootstrapped replica S_t by randomly drawing F percent of S .
2. Call **WeakLearn** with S_t and receive the hypothesis (classifier) h_t .
3. Add h_t to the ensemble, E .

End

Test: Simple Majority Voting – Given unlabeled instance $\mathbf{x}$

1. Evaluate the ensemble $E = \{h_1, \dots, h_T\}$ on $\mathbf{x}$.
2. Let $v_{t,j} = \begin{cases} 1, & \text{if } h_t \text{ picks class } \omega_j \\ 0, & \text{otherwise} \end{cases} \quad (8)$

be the vote given to class ω_j by classifier h_t .

3. Obtain total vote received by each class

$$V_j = \sum_{t=1}^T v_{t,j}, \quad j = 1, \dots, C \quad (9)$$

4. Choose the class that receives the highest total vote as the final classification.

From Bagging → Random Forest

- Bagging:
 - Bootstrap samples
 - Same feature set
- Random Forest:
 - Bootstrap samples
 - Random feature selection at each split

Random Forest Algorithm

For $t = 1$ to T :

1. Sample data with replacement $\rightarrow S_t$

2. Train decision tree:

At each split:

randomly select m features

choose best split among m

3. Add tree h_t to ensemble

Prediction:

Majority vote (classification)

Average (regression)

Why Random Forest Works Better

- Reduces **correlation between trees**
- Strong variance reduction
 - Variance reduction works best when models are less correlated

prediction = true signal + error

Average N trees: $\text{Ensemble} = f(x) + \frac{1}{N} \sum \epsilon_i$

All trees make **similar errors**: $\epsilon_1 \approx \epsilon_2 \approx \dots \approx \epsilon_N$

So averaging gives: $\frac{1}{N}(\epsilon + \epsilon + \dots) = \epsilon$

No error reduction!

Why Random Forest Works Better

Variance of the average:

$$\text{Var}(\text{ensemble}) = \frac{1}{N}\sigma^2 + \frac{N-1}{N}\rho\sigma^2$$

σ^2 = variance of individual models

ρ = correlation between models

- If $\rho = 1$ (fully correlated):
variance = $\sigma^2 \rightarrow$ no improvement
- If $\rho = 0$ (uncorrelated):
variance = $\sigma^2/N \rightarrow$ huge reduction
- Handles high-dimensional data well

Key Parameters

- `n_estimators` (T)
- `max_features` (m)
- `max_depth`
- `min_samples_leaf`

Boosting

- One of the most important developments in the field of machine learning.
- Freund and Schapire, 1997
- Many variations
 - General boosting
 - Adaboost
 - Adaboost.R
 - Adaboost.M1

Core Idea of Boosting

- Boosting builds a strong model by adding weak models that fix residual errors.
 - Train models **sequentially**
 - Focus on **previous errors**
 - Combine with **weights**
- Instead of averaging, it **corrects mistakes step-by-step**
 - The model becomes **more expressive**
 - It can fit complex patterns

General Boosting

- Basic ideas: create three weak classifiers:
 - classifier C1 trained with a random subset of the available training data.
 - C2 is trained on a training data only half of which is correctly classified by C1 ,and the other half is misclassified.
 - The third classifier C3 is trained with instances on which C1 and C2 disagree.
 - The three classifiers are combined through a three-way majority vote.
- No replacement allowed

General Boosting

Algorithm: Boosting

Input:

- Training data S of size N with correct labels $\omega_i \in \Omega = \{\omega_1, \omega_2\}$;
- Weak learning algorithm **WeakLearn**.

Training

1. Select $N_1 < N$ patterns without replacement from S to create data subset S_1 .
2. Call **WeakLearn** and train with S_1 to create classifier C_1 .
3. Create dataset S_2 as the most informative dataset, given C_1 , such that half of S_2 is correctly classified by C_1 , and the other half is misclassified. To do so:
 - a. Flip a fair coin. If Head, select samples from S , and present them to C_1 until the first instance is misclassified. Add this instance to S_2 .

- b. If Tail, select samples from S , and present them to C_1 until the first one is correctly classified. Add this instance to S_2 .
 - c. Continue flipping coins until no more patterns can be added to S_2 .
4. Train the second classifier C_2 with S_2 .
 5. Create S_3 by selecting those instances for which C_1 and C_2 disagree. Train the third classifier C_3 with S_3 .

Test – Given a test instance $\mathbf{x}$

1. Classify $\mathbf{x}$ by C_1 and C_2 . If they agree on the class, this class is the final classification.
2. If they disagree, choose the class predicted by C_3 as the final classification.

AdaBoost Intuition

- Misclassified points $\rightarrow$ higher weight
- Next model focuses on “hard” cases

AdaBoost

- Basic ideas:
 - Consecutive classifiers' training data are geared towards increasingly hard-to-classify instances.
 - How?
 - Train each weak classifier using instances drawn from an iteratively updated distribution of the training data.
 - This distribution update ensures that instances misclassified by the previous classifier are more likely to be included in the training data of the next classifier.
 - Weighted majority vote

AdaBoost Example

- Training set: $S = \{(x_i, y_i)\}_{i=1}^N$
- Initial weights: $D_1(i) = \frac{1}{N}$

$i \Rightarrow$	0_1	0_2	0_3	0_4	0_5	0_6	0_7	0_8	0_9	0_{10}
D_1	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$

Iteration 1: Train First Classifier

- Train h_1
 - Train weak learner using D_1
 - Some points are misclassified

$i \Rightarrow$	o_1	o_2	o_3	o_4	o_5	o_6	o_7	o_8	o_9	o_{10}
D_1	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$
h_1			x	x	x				x	

Compute Error

- Error is computed with:

$$\epsilon_t = \sum_{i: h_t(x_i) \neq y_i} D_t(i)$$

- Error < 0.5 $\rightarrow$ acceptable weak learner

$i \Rightarrow$	0_1	0_2	0_3	0_4	0_5	0_6	0_7	0_8	0_9	0_{10}
D_1	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$
h_1			x	x	x				x	
misclassified ($h_1(0_i) \neq y_i$)										
$\epsilon_1 =$	$\frac{1}{10} + \frac{1}{10} + \frac{1}{10} + \frac{1}{10} = \frac{4}{10}$									

Compute Model Weight

- Model weight is computed as:
 - Smaller error $\rightarrow$ smaller β
- Confidence = $\log(1/\beta)$
 - Smaller $\beta \rightarrow$ **higher confidence**

$$\beta_t = \frac{\epsilon_t}{1 - \epsilon_t}$$

$i \Rightarrow$	O_1	O_2	O_3	O_4	O_5	O_6	O_7	O_8	O_9	O_{10}
D_1	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$
h_1			x	x	x				x	
misclassified ($h_1(O_i) \neq y_i$)										
$\epsilon_1 =$	$\frac{1}{10} + \frac{1}{10} + \frac{1}{10} + \frac{1}{10} = \frac{4}{10}$					$\beta_1 = \frac{\epsilon_1}{(1-\epsilon_1)} = \frac{(\frac{4}{10})}{(\frac{6}{10})} = \frac{2}{3}$ $\frac{1}{\beta_1} = \frac{3}{2}$ (less confidence)				

Update Weights (Important Step)

- Weight of each data is updated with:

$$D_2(i) = \frac{D_1(i)}{Z_1} \times \begin{cases} \beta_1 & \text{if correct} \\ 1 & \text{if wrong} \end{cases}$$

- Correct $\rightarrow$ weight drops
- Wrong $\rightarrow$ weight increases

i =>	O ₁	O ₂	O ₃	O ₄	O ₅	O ₆	O ₇	O ₈	O ₉	O ₁₀	
D ₁	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	
h ₁			x	x	x				x		
misclassified (h1(O _i) ≠ y _i)											
ε ₁ =	$\frac{1}{10} + \frac{1}{10} + \frac{1}{10} + \frac{1}{10} = \frac{4}{10}$					$\beta_1 = \frac{\varepsilon_1}{(1-\varepsilon_1)} = \frac{(\frac{4}{10})}{(\frac{6}{10})} = \frac{2}{3}$					
	$\frac{1}{\beta_1} = \frac{3}{2}$ (less confidence)										
D ₂	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{10}$	$\frac{1}{15}$	

Iteration 2

- Repeat the steps:
 - Train h_2 using D_2
 - Correctly classified objects from previous iteration is less likely selected to training data in the 2nd iteration
 - Compute: ϵ_2, β_2
 - ϵ_2 smaller $\rightarrow$ better classifier
 - β_2 smaller $\rightarrow$ higher weight

Results After Iteration 2

i =>	O ₁	O ₂	O ₃	O ₄	O ₅	O ₆	O ₇	O ₈	O ₉	O ₁₀
D ₁	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$
h ₁			x	x	x				x	
misclassified (h1(O _i) ≠ y _i)										
ε ₁ =	$\frac{1}{10} + \frac{1}{10} + \frac{1}{10} + \frac{1}{10} = \frac{4}{10}$ $\beta_1 = \frac{\varepsilon_1}{(1-\varepsilon_1)} = \frac{\left(\frac{4}{10}\right)}{\left(\frac{6}{10}\right)} = \frac{2}{3}$ $\frac{1}{\beta_1} = \frac{3}{2}$ (less confidence)									
D ₂	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{10}$	$\frac{1}{15}$
Normalize	$\frac{1}{12}$	$\frac{1}{12}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{12}$	$\frac{1}{12}$	$\frac{1}{12}$	$\frac{1}{8}$	$\frac{1}{12}$
		O ₂	O ₃	O ₄		O ₆	O ₇		O ₉	
h ₂				x						
ε ₂ =	$\frac{1}{8}$ $\beta_2 = \frac{\varepsilon_2}{1-\varepsilon_2} = \frac{\frac{1}{8}}{\frac{7}{8}} = \frac{1}{7}$ $\frac{1}{\beta_2} = 7$ (more confidence)									

What's happening?

- Hard points keep getting heavier
- Easy points fade away
- Model focuses on difficult cases
- The algorithm zooms in on mistakes!

After Iteration 3

i =>	O ₁	O ₂	O ₃	O ₄	O ₅	O ₆	O ₇	O ₈	O ₉	O ₁₀	
D ₁	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	
h ₁			x	x	x				x		
misclassified (h1(O _i) ≠ y _i)											
ε ₁ =	$\frac{1}{10} + \frac{1}{10} + \frac{1}{10} + \frac{1}{10} = \frac{4}{10}$ $\beta_1 = \frac{\epsilon_1}{(1-\epsilon_1)} = \frac{(\frac{4}{10})}{(\frac{6}{10})} = \frac{2}{3}$ $\frac{1}{\beta_1} = \frac{3}{2}$ (less confidence)										
D ₂	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{10}$	$\frac{1}{15}$	
Normalize	$\frac{1}{12}$	$\frac{1}{12}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{12}$	$\frac{1}{12}$	$\frac{1}{12}$	$\frac{1}{8}$	$\frac{1}{12}$	
		O ₂	O ₃	O ₄		O ₆	O ₇		O ₉		
h ₂				x							
ε ₂ =	$\frac{1}{8}$ $\beta_2 = \frac{\epsilon_2}{1-\epsilon_2} = \frac{\frac{1}{8}}{\frac{7}{8}} = \frac{1}{7}$ $\frac{1}{\beta_2} = 7$ (more confidence)										
D ₃	$\frac{1}{12}$	$\frac{1}{12} * \frac{1}{7}$	$\frac{1}{8} * \frac{1}{7}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{12} * \frac{1}{7}$	$\frac{1}{12} * \frac{1}{7}$	$\frac{1}{12}$	$\frac{1}{8} * \frac{1}{7}$	$\frac{1}{12}$	
	$\frac{1}{12}$	$\frac{1}{84}$	$\frac{1}{56}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{84}$	$\frac{1}{84}$	$\frac{1}{12}$	$\frac{1}{56}$	$\frac{1}{12}$	
Normalize D ₃ :	$\frac{588}{7056}$	$\frac{84}{7056}$	$\frac{126}{7056}$	$\frac{441}{7056}$	$\frac{441}{7056}$	$\frac{84}{7056}$	$\frac{84}{7056}$	$\frac{588}{7056}$	$\frac{126}{7056}$	$\frac{588}{7056}$	

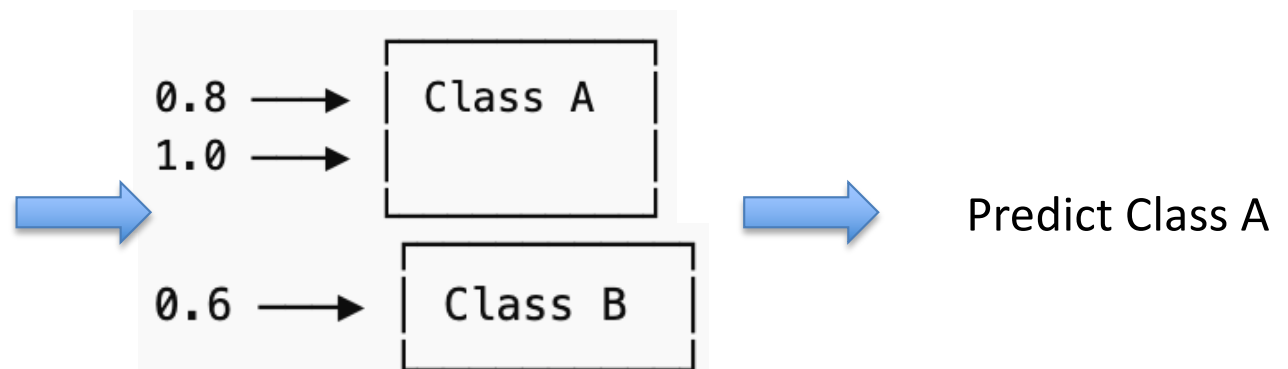
Final Prediction

- Each classifier $h_t(x)$ votes
- Stronger classifiers $\rightarrow$ bigger vote
- Choose the class V_j that received the highest vote

$$V_j = \sum_t \log \left(\frac{1}{\beta_t} \right) \cdot \mathbf{1}(h_t(x) = \omega_j)$$

$$H(x) = \arg \max_j V_j$$

$h_1(x) \rightarrow$ Class A weight = 0.8
 $h_2(x) \rightarrow$ Class B weight = 0.6
 $h_3(x) \rightarrow$ Class A weight = 1.0



AdaBoost

Algorithm AdaBoost.M1

Input:

- Sequence of N examples $S = [(\mathbf{x}_i, y_i)], i = 1, \dots, N$ with labels $y_i \in \Omega, \Omega = \{\omega_1, \dots, \omega_C\}$;
- Weak learning algorithm **WeakLearn**;
- Integer T specifying number of iterations.

Initialize $D_1(i) = \frac{1}{N}, i = 1, \dots, N$ (11)

Do for $t = 1, 2, \dots, T$:

1. Select a training data subset S_t , drawn from the distribution D_t .
2. Train **WeakLearn** with S_t , receive hypothesis h_t .

3. Calculate the error of h_t : $\varepsilon_t = \sum_{i: h_t(\mathbf{x}_i) \neq y_i} D_t(i)$. (12)

If $\varepsilon_t > 1/2$, **abort**.

4. Set $\beta_t = \varepsilon_t / (1 - \varepsilon_t)$. (13)

5. Update distribution

$$D_t : D_{t+1}(i) = \frac{D_t(i)}{Z_t} \times \begin{cases} \beta_t & \text{if } h_t(\mathbf{x}_i) = y_i \\ 1, & \text{otherwise} \end{cases} \quad (14)$$

where $Z_t = \sum_i D_t(i)$ is a normalization constant chosen so that D_{t+1} becomes a proper distribution function.

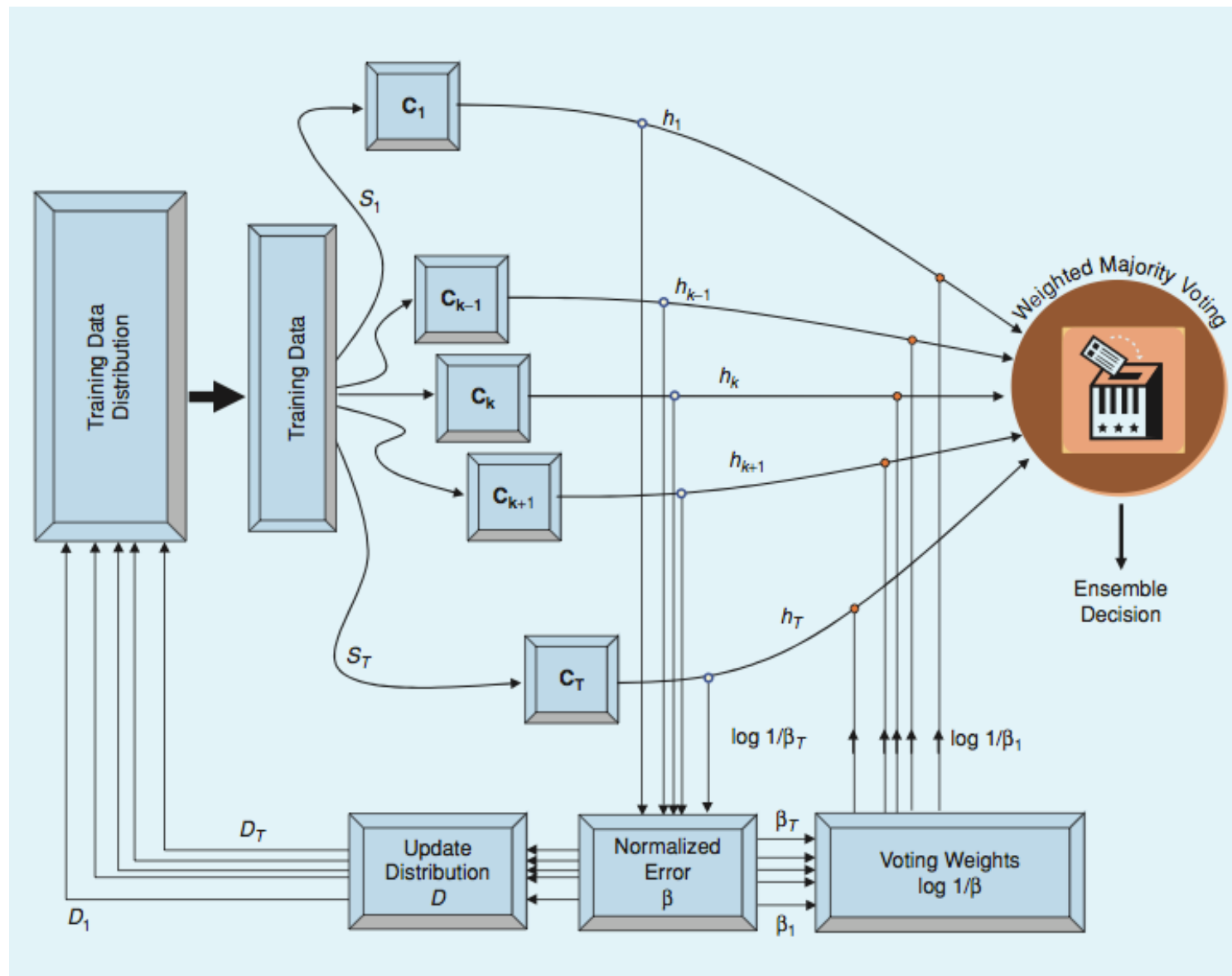
Test – Weighted Majority Voting: Given an unlabeled instance $\mathbf{x}$,

1. Obtain total vote received by each class

$$V_j = \sum_{t: h_t(\mathbf{x}) = \omega_j} \log \frac{1}{\beta_t}, j = 1, \dots, C. \quad (15)$$

2. Choose the class that receives the highest total vote as the final classification.

Adaboost



Boosting

- It has been proven: “the error of this three-classifier ensemble is bounded above, and it is less than the error of the best classifier in the ensemble, provided that each classifier has an error rate that is less than 0.5.”

Combining Classifiers

- A strategy to combine the output of individual classifiers → amplify correct decisions and cancel out incorrect ones:
 - Majority vote
 - Weighted majority vote
 - Combining numeric outputs
 - Others like Borda count (rank-based voting)

When to Use What

- Use Random Forest when:
 - Structured (tabular) data without spatial/sequence structure
 - No need for feature scaling, embeddings, handles mixed feature types
 - Need a **reliable, low-tuning model**
 - Works well on many datasets without much tuning
 - Handles nonlinear relationships, feature interactions
 - Robust to noise, scaling issues
 - Few hyperparameters matter critically
- Use Boosting when:
 - Want highest accuracy
 - Data is clean